

ML hw5 Report

I. Gaussian Process

1. Codes

formula of rational quadratic kernel

```
def rational_quadratic_kernel(A, B, params):
    """
    Computes the Rational Quadratic (RQ) kernel matrix between two input matrices A and B.
    params: Log-transformed hyperparameters [log(sigma_f^2), log(l), log(alpha)].
    """
    # Convert log parameters back to original scale
    sigma_f_sq = np.exp(params[0])
    l = np.exp(params[1])
    alpha = np.exp(params[2])

    # Calculate the squared Euclidean distance matrix
    dist_sq = cdist(A, B, metric='sqeuclidean')

    # Apply the Rational Quadratic kernel formula: K = sigma_f^2 * (1 + dist^2 / (2 * alpha * l^2))^-alpha
    K = sigma_f_sq * (1.0 + dist_sq / (2.0 * alpha * l**2))**(-alpha)
    return K
```

The code implements the Rational Quadratic ($k(x_i, x_j) = \left(1 + \frac{d(x_i, x_j)^2}{2\alpha l^2}\right)^{-\alpha}$) Kernel, which allows the Gaussian Process to model data with varying length-scales (a mixture of squared-exponential kernels).

The function takes the input matrices and hyperparameters (signal variance σ_f^2 , length-scale l , and mixture parameter α). Note that the parameters are passed in log-space to ensure positivity during optimization, so we first exponentiate them.

We then compute the squared Euclidean distance between inputs and apply the RQ formula shown below. This kernel matrix K is subsequently used in the Log-Likelihood function to optimize these hyperparameters."

Optimize the kernel parameters

```
def negative_log_likelihood(log_params, X_train, y_train, beta_inv):
    """
    Calculates the Negative Marginal Log-Likelihood (NMLL) for the GP model.
    This is the objective function for optimization.
    """
    N = len(X_train)

    # Compute the kernel matrix K(X, X)
    K = rational_quadratic_kernel(X_train, X_train, log_params)

    # Construct the covariance matrix C = K + beta^-1 * I (I is the identity matrix)
    C = K + beta_inv * np.eye(N)

    try:
        C_inv = inv(C)

        # Marginal Log-Likelihood (MLL) formula
        log_likelihood = - 0.5 * np.log(det(C)) - 0.5 * y_train.T @ C_inv @ y_train - N/2 * np.log(2.0 * np.pi)

        return -log_likelihood.flatten()[0]

    except LinAlgError:
        # Return a large penalty value if Cholesky decomposition fails (C is not positive definite)
        return 1e10
```

```
def optimize_gp_params(X, y, beta_inv):
    """
    Minimizes the NMLL using scipy.optimize.minimize to find the optimal kernel hyperparameters.
    """
    # Set initial log parameters: [log(sigma_f^2), log(1), log(alpha)]
    initial_log_params = np.log(np.array([1.0, 1.0, 1.0]))

    # Set boundaries for log parameters
    bounds = [(-5.0, 5.0), (-5.0, 5.0), (-5.0, 5.0)]

    # Run the L-BFGS-B optimization algorithm
    result = minimize(
        fun=negative_log_likelihood,
        x0=initial_log_params,
        args=(X, y, beta_inv),
        method='L-BFGS-B',
        bounds=bounds
    )

    # Convert optimized log parameters back to original scale
    optimized_params = np.exp(result.x)

    # Return results
    return optimized_params, result.fun, result.success
```

To find the optimal kernel parameters, we define the objective function using the Marginal Log-Likelihood of the Gaussian Process. The probability density is given by:

$$p(\mathbf{y}|\theta) = \mathcal{N}(\mathbf{y}|\mathbf{0}, \mathbf{C}_\theta)$$

Taking the logarithm gives us the log-likelihood function:

$$\ln p(\mathbf{y}|\theta) = -\frac{1}{2}\ln |\mathbf{C}_\theta| - \frac{1}{2}\mathbf{y}^\top \mathbf{C}_\theta^{-1} \mathbf{y} - \frac{N}{2}\ln(2\pi)$$

In our code, the covariance matrix $\mathbf{C}_\theta$ is constructed as $\mathbf{K} + \text{beta_inv} * \mathbf{I}$, where $\mathbf{K}$ is the Rational Quadratic kernel matrix and beta_inv represents the noise variance (β^{-1}).

Since standard optimization libraries like `scipy.optimize` are designed to **minimize** cost functions, we implement the **Negative Log-Likelihood**. By minimizing the negative log-likelihood, we effectively maximize the probability of the observed data given the parameters.

predict the distribution

```
def gp_predict(X_train, y_train, X_test, kernel_params, beta_inv):
    """
    Performs Gaussian Process prediction on test points, returning the mean and variance.
    """
    N = len(X_train)
    # Compute the covariance matrix C and solve for alpha = C^-1 y
    K = rational_quadratic_kernel(X_train, X_train, np.log(kernel_params))
    C = K + beta_inv * np.eye(N)
    alpha = solve(C, y_train)

    mu_star = []
    sigma_f_sq_star = []

    # Predict mean and variance for each test point x*
    for x_star in X_test:
        x_star = x_star.reshape(1, -1)
        # k* = K(X_train, x*)
        k_star = rational_quadratic_kernel(X_train, x_star, np.log(kernel_params))

        # Predicted Mean: mu(x*) = k*^T * alpha
        mu = k_star.T @ alpha
        mu_star.append(mu.flatten()[0])

        # Predicted Variance: sigma_f^2(x*) = k(x*, x*) - k*^T C^-1 k*
        k_star_star = rational_quadratic_kernel(x_star, x_star, np.log(kernel_params))
        v = solve(C, k_star) # v = C^-1 k*
        sigma_sq = k_star_star - k_star.T @ v
        sigma_f_sq_star.append(sigma_sq.flatten()[0])

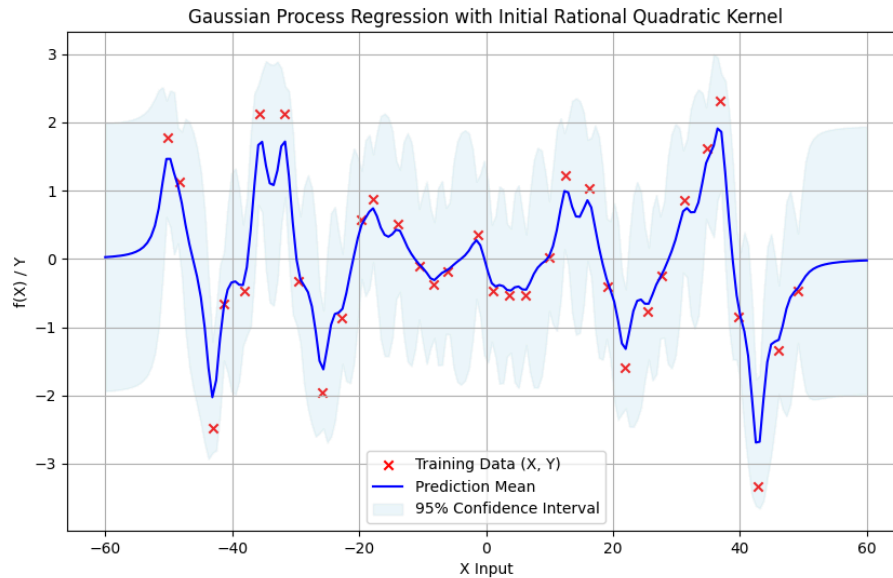
    return np.array(mu_star), np.array(sigma_f_sq_star)
```

To perform prediction on the test data $\mathbf{x}_{\text{test}}$, we compute the posterior mean and variance for each test point $\mathbf{x}^*$.

We iterate through each test point to calculate the kernel vector $\mathbf{k}_*$ between the training data and the test point. The predictive mean $\mu(\mathbf{x}^*)$ represents the regression line, while the variance $\sigma^2(\mathbf{x}^*)$ represents the uncertainty of the latent function (used for the confidence interval)."

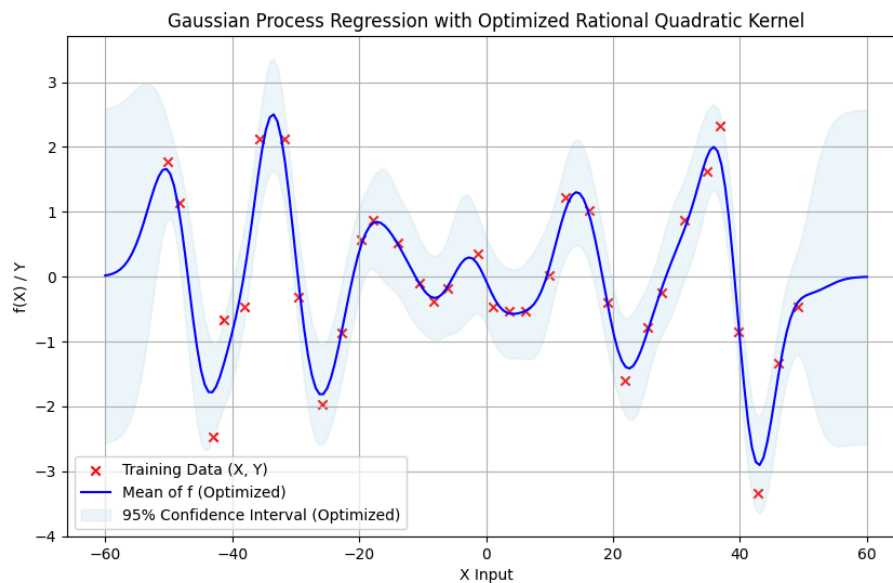
2. Experiments

Task1



Given the parameters $\sigma_f^2 = 1.0$, $l = 1.0$, $\alpha = 1.0$, the regression doesn't approximate the data points perfectly.

Task2



The optimized result is shown as the picture. Note that the optimized solution gives $\sigma_f^2 = 1.73031937$, $l = 3.30971066$, $\alpha = 50$ and the marginal

log-likelihood estimation is 50.7280

3. Observation and Discussion

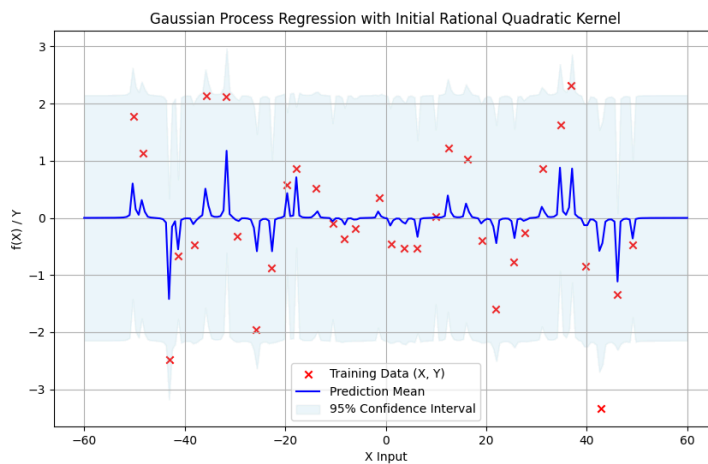
Length Scale of the Rational Quadratic Kernel

The Rational Quadratic kernel is defined as:

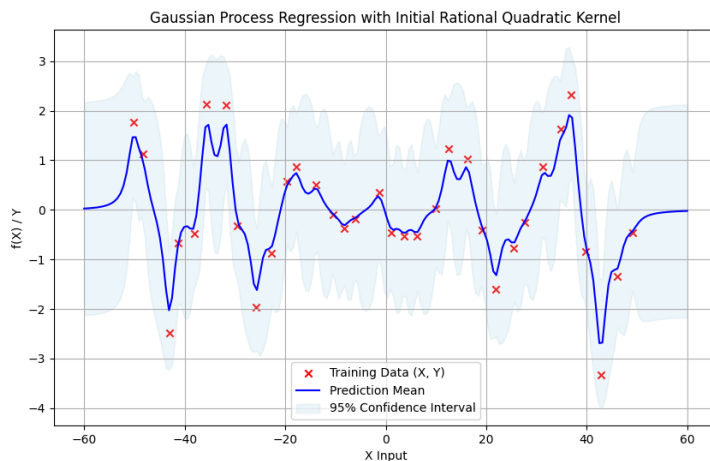
$$k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \left(1 + \frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\alpha l^2} \right)^{-\alpha}$$

The parameter l (**length scale**) determines the "smoothness" of the function. It defines how far apart two input points can be while still being considered correlated.

- **Small l :** The correlation decays rapidly with distance. This allows the model to capture rapid variations and high-frequency details, but it risks **overfitting** (interpreting noise as signal).
 - *Example ($l = 0.1$):* The prediction appears jagged and tries to pass through every training point strictly.



- **Large l :** The correlation decays slowly. The model assumes the underlying function is very smooth.
 - *Example ($l = 1.0$):* The prediction creates a smooth curve that averages out local variations.



Choosing a proper l is crucial to balance between capturing the trend and ignoring the noise.

II. SVM on MNIST

1. Codes

Task1

```
def train_and_predict_svm(X_train, Y_train, X_test, Y_test, kernel_type='linear', C=1.0, gamma='auto'):
    # Simulates SVM training and prediction using LIBSVM (or Scikit-learn SVC).
    print(f"> Training with Kernel: {kernel_type}, C={C}, gamma={gamma}")

    # 1. Build LIBSVM parameter string
    param_list = ['-s 0'] # -s 0: C-SVC

    # Core: Set kernel type (-t) and related parameters
    if kernel_type == 'linear':
        param_list.append('-t 0')
    elif kernel_type == 'poly':
        param_list.append('-t 1 -d 3') # Default degree 3 (-d 3)
    elif kernel_type == 'rbf':
        param_list.append('-t 2')
        param_list.append(f'-g {gamma}')
    elif kernel_type == 'precomputed':
        param_list.append('-t 4')

    param_list.append(f'-c {C}')
    param_list.append('-q') # Quiet mode

    param_str = ' '.join(param_list)

    # 2. train model
    model = svm_train(Y_train, X_train, param_str)

    # 3. Predict and get accuracy
    p_label, p_acc, p_val = svm_predict(Y_test, X_test, model)

    accuracy = p_acc[0]
    return accuracy
```

The `train_and_predict_svm` function serves as a wrapper for the LIBSVM library. Its primary goal is to map high-level configuration strings (like 'linear' or 'rbf') into the specific command-line flags required by LIBSVM.

We construct the parameter string `param_str` to define the kernel type using the `-t` flag:

- **Linear (-t 0):** $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$
- **Polynomial (-t 1):** $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^\top \mathbf{x}_j + r)^d$
- **RBF (-t 2):** $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$

Additionally, we pass the regularization parameter **Cost** (C) via the `-c` flag and the kernel coefficient **Gamma** (γ) via the `-g` flag. Finally, the function trains the model and returns the prediction accuracy on the test set.

Task2

```
def comprehensive_grid_search(X, y, folds=5):
    # Systematically searches (Grid Search) C, Gamma, and Degree for Linear, Poly, and RBF kernels.

    # 1. Define hyperparameter search ranges
    C_range = [0.1, 1, 10, 100] # Penalty parameter C
    Gamma_range = [0.0001, 0.001, 0.01, 0.1] # Gamma for RBF and Poly kernels
    Degree_range = [2, 3] # Degree d for Polynomial kernel

    overall_best_acc = -1.0
    overall_best_config = {}

    print("===== Starting Comprehensive Kernel Grid Search =====")

    # --- 2. Linear Kernel (-t 0): Search C only ---
    print("\n--- Searching Linear Kernel (-t 0) ---")

    for C in C_range:
        param_str = f'-s 0 -t 0 -c {C} -v {folds} -q'

        # Performs K-fold cross-validation
        cv_acc = svm_train(y, X, param_str)

        print(f"Linear: C={C:.2e} -> CV Acc = {cv_acc:.4f}%")
        config = {'kernel': 'linear', 'C': C, 'acc': cv_acc}

        if cv_acc > overall_best_acc:
            overall_best_acc = cv_acc
            overall_best_config = config
```

To optimize the performance of the Soft-Margin SVM, we implement a **Grid Search** strategy to systematically test combinations of hyperparameters. We define search ranges for:

- **Penalty (C):** Controls the trade-off between decision boundary smoothness and classifying training points correctly.
- **Gamma (γ) and Degree (d):** Parameters specific to the RBF and Polynomial kernels.

We evaluate each configuration using **K -Fold Cross-Validation** (specified by the `-v` flag in LIBSVM). Instead of returning a trained model, enabling `-v` instructs LIBSVM to split the training data into K folds (in this case, 5), train on $K - 1$ folds, and validate on the remaining fold.

The function iterates through all parameter combinations, records the cross-validation accuracy returned by `svm_train`, and stores the configuration that yields the highest accuracy.

Task3

```

# --- Task 3: Custom Kernel K(x_i, x_j) ---

def linear_rbf_kernel_matrix(X1, X2, C_rbf, gamma):
    # Computes the custom kernel matrix: K_new = K_linear + C_rbf * K_RBF.

    # 1. Compute Linear Kernel: K_linear = X1 * X2.T
    K_linear = X1 @ X2.T

    # 2. Compute RBF Kernel: K_RBF = exp(-gamma * ||x_i - x_j||^2)
    dist_sq = cdist(X1, X2, metric='sqeuclidean')
    K_RBF = np.exp(-gamma * dist_sq)

    # 3. Combine the new kernel function
    K_new = K_linear + C_rbf * K_RBF

    return K_new

# --- Task 3 Core LIBSVM Interface Function ---

def train_and_predict_custom_svm(X_train, Y_train, X_test, Y_test, C, gamma, C_rbf=1.0):
    # Trains and predicts an SVM using the precomputed custom kernel (-t 4).

    N_train = X_train.shape[0]
    N_test = X_test.shape[0]

    print(f"> Training with Custom Kernel (Linear + RBF). C={C}, gamma={gamma}, C_rbf={C_rbf}")

    # 1 Train Data Formatting
    K_train_new = linear_rbf_kernel_matrix(X_train, X_train, C_rbf, gamma)

    # 2 Train Model
    param_str = f'-s 0 -t 4 -c {C} -g {gamma}'
    model = svm_train(Y_train, K_train_new, param_str)

    # 3 Test Data
    K_test_new = linear_rbf_kernel_matrix(X_test, X_train, C_rbf, gamma)

    p_label, p_acc, p_val = svm_predict(Y_test, X_test, model)

    accuracy = p_acc[0]

    return accuracy

```

To combine the Linear and RBF kernels, we define the custom kernel formula:

$$K'(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j) + C_{rbf} \cdot \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$$

We use the LIBSVM option `-t 4` (precomputed kernel), which allows `svm_train` to accept our calculated kernel matrix as input instead of raw features. Finally, we evaluate the model's performance on the test data using `svm_predict`."

2. Experiments

Task 1

- Training with Kernel: linear, $C=1.0$
Accuracy = 95.08% (2377/2500) (classification)
- Training with Kernel: poly, $C=1.0$
Accuracy = 34.68% (867/2500) (classification)
- Training with Kernel: rbf, $C=1.0$, $\gamma=0.001$
Accuracy = 95.24% (2381/2500) (classification)

Task 2

Linear kernel

Penalty C	Cross Validation Accuracy
C=1.00e-01	96.74%
C=1.00e+00	96.02%
C=1.00e+01	96.24%
C=1.00e+02	96.32%

RBF kernel

Penalty C	gamma g	Cross Validation Accuracy
1.00e-01	1.00e-04	79.54%
1.00e-01	1.00e-03	92.5%
1.00e-01	1.00e-02	96.46%
1.00e-01	1.00e-01	53.72%

Penaly C	gamma g	Cross Validation Accuracy
1.00e+00	1.00e-04	92.54%
1.00e+00	1.00e-03	96.16%
1.00e+00	1.00e-02	97.84%
1.00e+00	1.00e-01	91.96%

Penaly C	gamma g	Cross Validation Accuracy
1.00e+01	1.00e-04	96.14%
1.00e+01	1.00e-03	97.14%
1.00e+01	1.00e-02	98.18%
1.00e+01	1.00e-01	92.12%

Penaly C	gamma g	Cross Validation Accuracy
1.00e+02	1.00e-04	97.12%
1.00e+02	1.00e-03	97.32%
1.00e+02	1.00e-02	98.32%
1.00e+02	1.00e-01	92.2%

Polynomial kernel

penalty C	d	gamma g	Cross Validation Accuracy
1.00e-01	2	1.00e-04	45.52%
1.00e-01	2	1.00e-03	45.44%
1.00e-01	2	1.00e-02	94.72%
1.00e-01	2	1.00e-01	98%
1.00e-01	3	1.00e-04	28.16%
1.00e-01	3	1.00e-03	28.2%
1.00e-01	3	1.00e-02	90.2%
1.00e-01	3	1.00e-01	97.6%

Task 3

- Training with Custom Kernel (Linear + RBF). C=10, gamma=0.01, C_rbf=1.0
Accuracy = 20% (500/2500) (classification)

3. Observation and Discussion

I. Overfitting and Underfitting

Penaly C	gamma g	Cross Validation Accuracy
1.00e-01	1.00e-04	79.54%
1.00e-01	1.00e-03	92.5%
1.00e-01	1.00e-02	96.46%
1.00e-01	1.00e-01	53.72%

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \cdot \|\mathbf{x} - \mathbf{x}'\|^2)$$

The meaning of gamma g in rbf kernel is the length scale of the kernel. If gamma is large, the similarity decreasing in a fast speed. If gamma is small, the similarity decreasing slowly.

In other words, the SVM model can not capture the similarity of the data when gamma is $1.00e-04$, causing underfitting, while it cause overfitting when gamma equals $1.00e-01$

Moreover, the combination of $\gamma = 1.00e - 01$ (making the decision boundary extremely localized) and $C = 1.00e - 01$ (low penalty for errors) leads to severe overfitting to the training data. This overfit model has extremely poor generalization ability, causing the cross-validation accuracy to drop drastically to 53.72%.

II. Complexity vs. the length scale gamma g

penalty C	d	gamma g	Cross Validation Accuracy
1.00e-01	2	1.00e-04	45.52%
1.00e-01	2	1.00e-03	45.44%
1.00e-01	2	1.00e-02	94.72%
1.00e-01	2	1.00e-01	98%
1.00e-01	3	1.00e-04	28.16%
1.00e-01	3	1.00e-03	28.2%
1.00e-01	3	1.00e-02	90.2%
1.00e-01	3	1.00e-01	97.6%

The degree d in the Polynomial kernel is a fundamental hyperparameter that determines the maximum complexity (capacity) of the SVM model.

An interesting observation regarding the impact of hyperparameter interaction is evident:

1. When the degree is lower ($d = 2$), the extreme underfitting caused by the small γ value (10^{-4} or 10^{-3}) deteriorates performance to approximately
2. .However, when the degree is higher ($d = 3$), the same extreme underfitting caused by the small γ leads to a more catastrophic

performance failure, reducing the accuracy to approximately 28%.

This suggests that in conditions of severe constraint (low γ and low C), the $d = 3$ kernel's increased potential complexity fails more acutely than the simpler $d = 2$ kernel. The interaction of a very small scaling factor (γ) with a higher order feature transformation (d) results in a less effective data separation in the transformed space.